



Documentation de Conception

Projet Genie Logiciel - gl16

ENSIMAG 2025/2026

AIT MOHAMED Younes (aitmohay)

BEAURAIN Tom (beurato)

BEN ISMAIL Wajd (benismaw)

FURLANI DA SILVA SOARES Manuela (furlanim)

MARIA ENGLER PINTO Carlos (mariaenc)

Table des matières

1	Objectif du document	3
2	Vue d'ensemble de l'architecture	3
2.1	Chaîne de compilation	3
2.2	Découpage en modules (packages)	3
2.3	Rôle de <code>DecacCompiler</code>	3
3	Étape B : conception des vérifications contextuelles	4
3.1	But de l'étape B	4
3.2	Modèle de données contextuel	4
3.2.1	Table des symboles	4
3.2.2	Environnement des types : <code>EnvironmentType</code>	4
3.2.3	Environnement des expressions : <code>EnvironmentExp</code>	5
3.3	Organisation des passes (mode objet)	5
3.3.1	Passé 1 : Déclaration des Classes	5
3.3.2	Passé 2 : Vérification des Membres	5
3.3.3	Passé 3 : Vérification du Corps	6
3.4	Règles contextuelles spécifiques (exemples structurants)	6
3.4.1	Affectation et compatibilité	6
3.4.2	<code>return</code> et type de la méthode	6
3.4.3	Sélection <code>(.)</code> et accès <code>protected</code>	7
3.4.4	<code>instanceof</code> et <code>cast</code>	7
4	Étape C : conception de la génération de code	7
4.1	But de l'étape C	7
4.2	Architecture générale et gestion des ressources	7
4.2.1	Rôle central de <code>DecacCompiler</code>	8
4.2.2	Gestionnaires de ressources	8
4.3	Organisation du programme IMA	8
4.4	Gestion de la pile et convention d'appel	9
4.4.1	Blocs d'activation et dynamicité	9
4.4.2	Rôle du suivi de pile pour le <code>TSTO</code>	9
4.5	Saturation des registres et calcul dynamique du <code>TSTO</code>	9
4.5.1	Algorithme de saturation des registres (spill)	9
4.5.2	Calcul a posteriori du <code>TSTO</code>	10
4.6	Génération des expressions	10
4.6.1	Opérations arithmétiques et gestion de la pile	10
4.6.2	Sécurité et contrôle des erreurs	10
4.6.3	Court-circuit des opérateurs logiques	10
4.7	Support objet en génération de code	11
4.7.1	Représentation mémoire d'un objet	11
4.7.2	Construction et héritage des tables de méthodes (VTables)	11
4.7.3	Appel de méthode (dispatch dynamique)	11
4.7.4	Initialisation des objets et chaînage des <code>init</code>	12
4.7.5	<code>instanceof</code> et <code>cast</code> en code	12

5	Points d'évolution	12
5.1	Renforcer la robustesse des diagnostics ANTLR	12
5.2	Pistes d'optimisation	12
6	Bilan de la conception	13

1 Objectif du document

Cette documentation de conception s'adresse à un développeur souhaitant maintenir et/ou faire évoluer le compilateur **decac**. Elle présente l'organisation générale de l'implémentation, en se concentrant sur :

- la conception architecturale des étapes B (vérifications contextuelles) et C (génération de code) ;
- les algorithmes et structures de données spécifiques introduits par notre implémentation ;
- les points d'extension recommandés (où modifier, quoi ajouter, quels impacts).

Le document évite volontairement :

- les listings de code longs ;
- la liste détaillée des méthodes par classe (le Javadoc remplit ce rôle).

2 Vue d'ensemble de l'architecture

2.1 Chaîne de compilation

Du point de vue interne, la compilation suit un pipeline déterministe :

1. analyse lexicale et syntaxique (ANTLR) et construction de l'AST ;
2. étape B : vérifications contextuelles (typage, résolutions, décorations) ;
3. étape C : génération du code IMA à partir de l'AST décoré.

La compilation s'arrête à la première erreur bloquante, afin d'éviter une cascade de diagnostics secondaires et un état incohérent dans l'AST.

2.2 Découpage en modules (packages)

L'implémentation est structurée par responsabilités :

- `fr.ensimag.deca` : orchestration, options, compilation de fichiers, instance `DecacCompiler` ;
- `fr.ensimag.deca.syntax` : lexer/parser ANTLR, gestion d'erreurs localisées ;
- `fr.ensimag.deca.tree` : nœuds de l'AST, décompilation, vérifications, génération de code ;
- `fr.ensimag.deca.context` : types, définitions, environnements, sous-typage, règles de visibilité ;
- `fr.ensimag.deca.tools` : table des symboles, utilitaires d'affichage, gestion pile/labels (selon votre code).

Ce découpage impose un sens naturel des dépendances :

- `syntax` construit des objets `tree` ;
- `tree` utilise `context` pour vérifier et `DecacCompiler` pour générer du code ;
- `tools` fournit des services à tous les autres packages.

2.3 Rôle de DecacCompiler

`DecacCompiler` joue le rôle de contexte de compilation :

- accès à l'environnement de types et à la table des symboles ;

- accumulation du programme IMA (`IMAProgram`);
- accès aux options (`-n`, `-r`, `-d`, `-P`, `-w`, etc.);
- services partagés (création de labels uniques, allocation pile/variables globales, insertion de tests d'exécution).

L'objectif est d'éviter que l'AST manipule directement des détails d'infrastructure (labels globaux, ressources partagées, etc.), et de centraliser les choix de conventions d'appel, de registres et de stratégie de sécurité.

3 Étape B : conception des vérifications contextuelles

3.1 But de l'étape B

L'étape B vérifie qu'un programme syntaxiquement correct est sémantiquement valide :

- typage des expressions et instructions;
- résolution des identifiants et contrôle de portée;
- cohérence des déclarations (doublons, masquages, signatures);
- règles spécifiques à l'objet : hiérarchie, surcharge/rédéfinition, visibilité `protected`, usage de `this`, compatibilités `null`/classes, `instanceof`, `cast`, `return`.

Le résultat attendu est un AST décoré : chaque nœud important possède une information de type, et de nombreux identifiants sont reliés à une *définition* (variable, champ, méthode, classe). Ces décorations sont indispensables à l'étape C (génération de code fiable).

3.2 Modèle de données contextuel

3.2.1 Table des symboles

La `SymbolTable` assure l'interning des identifiants : deux occurrences d'un même nom partagent un objet symbole identique. Cela permet :

- des comparaisons rapides (par référence);
- une cohérence entre environnements;
- une représentation compacte des noms.

3.2.2 Environnement des types : `EnvironmentType`

`EnvironmentType` maintient :

- les types primitifs (`int`, `float`, `boolean`, `void`);
- les classes (`ClassDefinition`);
- la classe racine `Object` (mode objet).

Il fournit un prédicat de sous-typage, utilisé dans plusieurs règles :

- affectations et paramètres (compatibilité);
- conversions et retours;
- visibilité `protected` (contrôle d'accès en contexte objet);
- vérification de `instanceof`.

3.2.3 Environnement des expressions : `EnvironmentExp`

L'environnement `EnvironmentExp` gère les identifiants (variables locales, paramètres, champs, méthodes) via un chaînage parent. Pour ce faire, nous utilisons un dictionnaire `env` de type `HashMap` qui associe chaque symbole à sa définition (`ExpDefinition`). Cette structure permet de représenter la pile des portées : chaque nouveau bloc crée une instance pointant vers son `parentEnvironment`.

L'insertion via la méthode `declare` vérifie systématiquement l'absence de doublons dans le dictionnaire du niveau courant, levant une exception en cas de conflit. À l'inverse, la recherche d'un identifiant est réursive : elle interroge le dictionnaire local puis remonte la chaîne des parents, permettant ainsi de gérer naturellement le masquage (*shadowing*) et l'accès aux variables globales ou héritées.

Ce modèle correspond à la portée lexicale standard et permet une évolution simple (ajout d'un nouveau type de définition, d'un nouveau niveau de bloc, etc.).

3.3 Organisation des passes (mode objet)

L'architecture de l'étape de vérification contextuelle (`Contextual Analysis`) dans notre compilateur Deca repose sur une séparation claire en trois passes successives. Cette organisation est implémentée principalement dans les classes de l'arbre abstrait situées dans `fr.ensimag.deca.tree`, orchestrée par la classe `Program` et déléguée aux listes de déclarations.

Les classes clés impliquées sont :

- `Program` : Point d'entrée qui appelle séquentiellement les trois passes sur les classes.
- `ListDeclClass` : Itère sur toutes les classes définies.
- `DeclClass` : Implémente la logique spécifique pour chaque classe.
- `DeclMethod`, `DeclField`, `MethodBody` : Gèrent les membres et le corps.

Voici le détail de l'implémentation pour chaque passe :

3.3.1 Passe 1 : Déclaration des Classes

Objectif : Construire l'environnement des types en déclarant toutes les classes (et leurs super-classes) avant d'analyser leur contenu.

Implémentation : Cette passe est initiée par l'appel à `verifyListClass` dans `ListDeclClass`.

- Pour chaque `DeclClass`, la méthode `verifyClass` est invoquée.
- Elle vérifie que la super-classe (si elle existe) est bien déclarée dans l'`EnvironmentType` du compilateur.
- Elle crée le symbole et la définition de la classe courante.
- Elle ajoute la classe à l'environnement global.

Cette étape est cruciale pour permettre les références croisées entre classes (ex : une classe A peut faire référence à une classe B définie plus loin).

3.3.2 Passe 2 : Vérification des Membres

Objectif : Vérifier les signatures des membres (champs et méthodes) et construire l'environnement local de chaque classe en respectant l'héritage.

Implémentation : Initiée par `verifyListClassMembers` dans `ListDeclClass`. Dans `DeclClass.verifyClassMembers` :

- On récupère la définition de la classe et de sa super-classe.
- On lie l’environnement des membres de la classe courante à celui de la super-classe (pour hériter des méthodes et champs parents).
- On appelle `verifyListField` sur la liste des champs (`ListDeclField`) puis `verifyListMethod` sur la liste des méthodes (`ListDeclMethod`).
- **DeclMethod** : Vérifie que la signature de la méthode est valide et contrôle les règles de redéfinition (override). Si la méthode existe déjà dans la super-classe, les signatures doivent correspondre.
- **DeclField** : Vérifie le type du champ et s’assure qu’il ne masque pas illégalement un champ existant.

3.3.3 Passe 3 : Vérification du Corps

Objectif : Vérifier le corps des méthodes et l’initialisation des champs, maintenant que tous les types et signatures sont connus.

Implémentation : Initiée par `verifyListClassBody` dans `ListDeclClass`. Dans `DeclClass.verifyClassBody` :

- **Initialisation des champs** : `fields.verifyListFieldBody` vérifie que les expressions d’initialisation des champs sont valides et compatibles avec le type du champ.
- **Corps des méthodes** : `methods.verifyListMethodBody` délègue à chaque méthode la vérification de son corps.
- La classe `MethodBody` (via `verifyBody`) parcourt les instructions et les expressions de la méthode. Elle utilise l’environnement construit en Passe 2 pour valider les identificateurs, les appels de méthodes et les types.

3.4 Règles contextuelles spécifiques (exemples structurants)

3.4.1 Affectation et compatibilité

Règle générale :

- si les types sont identiques : OK ;
- si la conversion implicite `int` vers `float` est autorisée : insertion d’un nœud `ConvFloat` ;
- en mode objet : autoriser l’affectation d’une sous-classe vers une super-classe (via sous-typage) ;
- autoriser `null` vers tout type classe.

La justification de l’insertion `ConvFloat` est la simplification de l’étape C : le code généré n’a plus à deviner un cast implicite, il le voit dans l’AST.

3.4.2 return et type de la méthode

La vérification d’un `return` dépend du type de retour de la méthode courante :

- méthode `void` : `return` avec expression interdit ;
- méthode non `void` : expression obligatoire et compatible avec le type de retour.

Un point important pour la maintenance est le choix de la direction du test de compatibilité :

- on vérifie que le type de l'expression est compatible avec le type attendu ;
- en cas de sous-typage, une expression de sous-classe est compatible avec un retour de super-classe.

3.4.3 Sélection (.) et accès `protected`

La sélection `e.f` ou `e.m(...)` implique :

- typer `e` et vérifier que c'est un type classe ;
- retrouver `f` ou `m` dans l'environnement de membres de la classe (ou d'une super-classe) ;
- vérifier les règles de visibilité, en particulier `protected` :
 - un accès `protected` est autorisé uniquement dans un contexte de classe ;
 - la classe courante doit être dans la hiérarchie adéquate ;
 - l'objet utilisé pour accéder doit respecter la contrainte de compatibilité.

Ces conditions expliquent pourquoi la vérification a besoin à la fois de :

- `currentClass` ;
- la classe du receveur `e` ;
- la relation de sous-typage.

3.4.4 `instanceof` et `cast`

`instanceof` vérifie que le test est pertinent :

- le type à gauche doit être un type classe (ou `null`) ;
- le type cible doit être un type classe ;
- la relation de sous-typage doit permettre un test non absurde (selon la règle de Deca).

Le `cast` :

- peut être purement statique (si types compatibles) ;
- ou imposer un contrôle à l'exécution si le langage le prévoit (dépend de vos choix et du `-n`).

4 Étape C : conception de la génération de code

4.1 But de l'étape C

L'étape C traduit l'AST décoré en assembleur IMA. Elle doit respecter :

- la sémantique du langage Deca ;
- la convention d'appel choisie ;
- l'insertion des contrôles d'exécution (si `-n` n'est pas activé) ;
- les contraintes matérielles (nombre de registres `-r`, pile limitée, etc.).

4.2 Architecture générale et gestion des ressources

La génération de code repose sur une extension du patron de conception *Interprète*, dans lequel chaque nœud de l'Arbre de Syntaxe Abstraite (AST) porte sa propre logique de traduction en assembleur IMA. Cette approche permet une correspondance directe

entre la structure syntaxique du programme et le code généré, tout en conservant une forte modularité.

Afin de garantir la compatibilité avec la compilation parallèle (option `-P`), toute variable globale (`static`) a été proscrite dans l'étape C. Chaque fichier source est compilé à l'aide d'une instance distincte de `DecacCompiler`, assurant ainsi une isolation complète des états de compilation.

4.2.1 Rôle central de `DecacCompiler`

La classe `DecacCompiler` agit comme un conteneur de ressources partagé par l'ensemble des nœuds de l'AST. Elle centralise :

- l'instance unique de `IMAPProgram` associée au fichier compilé ;
- les gestionnaires de pile et de registres ;
- la création de labels uniques et l'insertion des tests d'exécution.

Ce choix permet d'éviter que les nœuds de l'AST manipulent directement des détails bas niveau (registres, offsets, labels globaux), et garantit une cohérence globale de la convention d'appel et des stratégies de sécurité.

4.2.2 Gestionnaires de ressources

Pour limiter la complexité du package `fr.ensimag.deca.tree`, nous avons introduit des classes utilitaires dédiées dans `fr.ensimag.deca.tools` qui centralisent la manipulation des ressources de la machine cible :

- **RegManager** : Gère l'allocation des registres banalisés (jusqu'à R_{15} par défaut). Sa particularité est la gestion automatique du **spill** : lorsque la limite fixée par l'option `-r` est atteinte, le gestionnaire génère automatiquement les instructions `PUSH` et `POP` pour sauvegarder temporairement les valeurs sur la pile, permettant ainsi des calculs complexes même avec un nombre restreint de registres.
- **StackManager** : Centralise la gestion des offsets de pile et de la zone globale (GB). Il permet d'allouer dynamiquement des adresses pour les variables (`RegisterOffset`) et suit en temps réel l'occupation de la pile (`maxStack`).
- **LabelManager** : Utilitaire de génération de labels uniques. Il assure qu'aucune collision ne survient entre les étiquettes de contrôle (boucles `while`, structures `if-then-else`) et les labels de gestion d'erreurs, en utilisant un compteur interne pour suffixer des préfixes sémantiques.

4.3 Organisation du programme IMA

La génération de code dans la classe `Program` suit une structure rigoureuse où l'ordre des instructions dans le fichier final est dicté par la gestion des ressources et la hiérarchie de classes :

- **Initialisation des VTables (Tables des méthodes)** : C'est la première étape du programme. Le compilateur génère manuellement la VTable de la classe `Object` (fondation de l'héritage), puis celle des classes utilisateur. Ces tables sont stockées en zone globale (GB) via le `StackManager`.
- **Code principal (*Main*)** : Le corps du programme utilisateur est généré à la suite des initialisations de tables. Il se termine systématiquement par l'instruction `HALT`.

- **Blocs de gestion d’erreurs** : Situés après le `HALT`, ces blocs ne sont exécutés que via des branchements conditionnels (`BOV`, `BEQ`). Chaque bloc affiche un message explicatif incluant la ligne de l’erreur avant d’interrompre l’exécution avec `ERROR`.
- **Corps des classes (*Methods Body*)** : Les définitions de méthodes et les initialiseurs de champs sont générés en fin de fichier. Comme ils ne sont accessibles que par des instructions `BSR` ou `BSR @`, ils sont isolés du flux d’exécution principal.

4.4 Gestion de la pile et convention d’appel

4.4.1 Blocs d’activation et dynamicité

Pour chaque méthode, un bloc d’activation est défini. Contrairement à une allocation statique, notre approche utilise le `StackManager` pour calculer dynamiquement l’occupation :

- **Paramètres et `this`** : Placés avec des offsets négatifs par rapport à la base locale `LB`.
- **Sauvegardes temporaires** : En cas de saturation des registres (*spill*), le `RegManager` sollicite la pile, augmentant ainsi dynamiquement la taille du bloc.

4.4.2 Rôle du suivi de pile pour le TSTO

Le calcul du TSTO (Test Stack Overflow) est l’un des points les plus critiques pour la robustesse. Notre `StackManager` maintient un compteur `maxStack` qui cumule :

1. L’espace nécessaire aux variables locales et globales.
2. L’espace maximal utilisé lors des évaluations d’expressions (empilements temporaires).
3. Les registres sauvegardés lors des appels de méthodes ou des débordements de registres (*spilling*).

Cette précision permet de générer un code IMA sûr, capable de détecter un manque de mémoire avant même que l’erreur ne survienne, tout en évitant de réserver un espace inutilement large.

4.5 Saturation des registres et calcul dynamique du TSTO

4.5.1 Algorithme de saturation des registres (*spill*)

Lorsque l’évaluation d’expressions complexes dépasse le nombre de registres disponibles fixé par l’option `-r`, un mécanisme de saturation (*spill*) est déclenché. Les valeurs intermédiaires sont alors temporairement sauvegardées sur la pile à l’aide d’instructions `PUSH` et restaurées via `POP`.

Le registre `R0` est utilisé comme registre pivot pour manipuler les valeurs restaurées depuis la pile avant leur recombinaison avec les registres banalisés. Ce choix permet d’éviter toute perte de données tout en conservant une stratégie d’allocation simple et déterministe.

4.5.2 Calcul a posteriori du TSTO

La valeur passée à l'instruction TSTO ne peut être connue qu'après la génération complète du corps d'une méthode ou du programme principal. Elle est calculée dynamiquement comme la somme :

$$d = nbSavedRegs + nbLocals + maxTempStack$$

L'instruction TSTO est insérée au début du bloc via un index de préambule mémorisé, ce qui permet de décaler le code généré sans modifier le parcours récursif de l'AST.

4.6 Génération des expressions

La génération des expressions repose sur une évaluation récursive des sous-arbres. Le résultat final de chaque expression est systématiquement placé dans un registre de destination fourni par le `RegManager`.

4.6.1 Opérations arithmétiques et gestion de la pile

Pour les opérations binaires (Plus, Minus, Divide), nous avons fait le choix d'une évaluation sécurisée par la pile afin de prévenir l'écrasement des résultats intermédiaires :

- **Sauvegarde systématique** : Le résultat du membre de gauche est empilé (PUSH) avant l'évaluation du membre de droite. Cette stratégie garantit l'intégrité de la valeur de gauche, même si le membre de droite contient des appels de méthodes ou des expressions complexes.
- **Registres de travail** : Après l'évaluation de la partie droite dans le registre de destination, la valeur de gauche est récupérée via un POP dans un registre scratch (R1). L'opération (ADD, SUB, DIV) est ensuite effectuée entre les registres scratch (R0, R1) avant d'être chargée dans le registre cible.
- **Optimisation des Identificateurs** : Dans le cas de la multiplication (Multiply), si le membre de gauche est un `Identifieur`, le compilateur optimise le code en évitant l'usage de la pile. Il génère directement une instruction de type `MUL addr, register`, utilisant l'adresse mémoire directe de la variable (`DAddr`).

4.6.2 Sécurité et contrôle des erreurs

Chaque opération arithmétique intègre des branchements vers les blocs d'erreurs définis dans `Program` :

- **Division** : Avant un `QUO` ou un `DIV`, une comparaison (`CMP`) vérifie la nullité du diviseur pour brancher vers `division_zero`.
- **Débordement** : Pour les calculs flottants, l'instruction `BOV` (*Branch on Overflow*) est systématiquement ajoutée après l'opération pour intercepter les dépassements de capacité (`debordement_flottant`).

4.6.3 Court-circuit des opérateurs logiques

Les opérateurs `And` et `Or` implémentent une évaluation paresseuse réelle via des branchements conditionnels :

- **And** : Si l'évaluation du membre gauche est fausse (BEQ), le programme saute directement au label `fin_and_i` sans évaluer la partie droite.
 - **Or** : Si le membre gauche est vrai (BNE), le programme saute au label `fin_or_i`.
- Cette gestion s'appuie sur le `LabelManager` pour garantir l'unicité des cibles de branchement dans le cas d'expressions booléennes imbriquées.

4.7 Support objet en génération de code

4.7.1 Représentation mémoire d'un objet

Un objet en mémoire (tas) est représenté par un bloc contigu contenant :

- à l'offset 0 : un pointeur vers la table des méthodes (VTable) de la classe ;
- à partir de l'offset 1 : les champs de l'objet, ordonnés selon la hiérarchie (champs de la super-classe suivis de ceux de la sous-classe).

Les offsets des champs sont calculés lors de l'étape B (décoration) et stockés dans les définitions, permettant à l'étape C de générer des instructions `LOAD/STORE` avec l'adressage `offset(Rn)` correct.

4.7.2 Construction et héritage des tables de méthodes (VTables)

Les VTables sont stockées dans la zone des variables globales (adressables via le registre GB). Lors de la génération de code pour une classe :

1. Une zone est allouée dans le segment global pour la VTable.
2. L'adresse de la VTable de la super-classe est copiée dans le premier slot.
3. Les adresses des méthodes sont ensuite remplies :
 - On recopie les méthodes héritées de la super-classe.
 - Si une méthode est redéfinie, son adresse écrase l'entrée existante (maintient du même index).
 - Les nouvelles méthodes sont ajoutées à la suite.

L'index de chaque méthode est fixé dès l'étape B, garantissant que pour une méthode donnée, son index est identique pour toute la hiérarchie, permettant le dispatch dynamique : `BSR offset(R2)`.

4.7.3 Appel de méthode (dispatch dynamique)

Pour un appel `o.m(args)` (`MethodCall.java`) :

1. On empile les arguments : allocation de place avec `ADDSP`, puis stockage des valeurs via `STORE` (les arguments sont empilés avant l'objet).
2. On évalue l'objet `o` et on stocke son adresse en sommet de pile.
3. Vérification de non-nullité (`CMP null`, `BEQ erreur`).
4. Chargement de l'adresse de la VTable depuis l'objet (offset 0).
5. Chargement de l'adresse de la méthode depuis la VTable (offset = index méthode).
6. Saut vers la méthode via `BSR`.
7. Au retour, nettoyage de la pile (`SUBSP`).

4.7.4 Initialisation des objets et chaînage des init

L'initialisation d'un objet instance d'une classe *D* respectant l'héritage suit un protocole strict :

1. allocation mémoire via `NEW` ;
2. mise à zéro des champs ;
3. appel récursif au sous-programme `init` de la super-classe ;
4. exécution des initialisations explicites propres à *D*.

Ce chaînage garantit qu'un champ hérité est toujours initialisé avant d'être potentiellement utilisé par une méthode appelée dans le constructeur de la classe fille.

4.7.5 instanceof et cast en code

L'opérateur `instanceof` et la vérification dynamique du `cast` reposent sur un algorithme identique d'inspection de la hiérarchie des types à l'exécution. La structure des VTables a été conçue pour faciliter ce parcours : chaque VTable contient à son offset 0 un pointeur vers la VTable de sa super-classe directe.

Concrètement, la vérification s'effectue par une boucle qui compare l'adresse de la VTable de l'objet inspecté avec celle de la classe cible. En cas de différence, l'algorithme remonte la chaîne d'héritage en chargeant la VTable parente (via l'offset 0). Si une correspondance est trouvée, le type est validé. Si la racine de la hiérarchie est atteinte (pointeur `null`) sans correspondance, le type est invalidé.

La différence réside uniquement dans l'issue du test :

- `instanceof` retourne un booléen : 1 en cas de succès, 0 en cas d'échec (ou si l'objet est `null`).
- Le `cast` est une assertion : il poursuit l'exécution en cas de succès mais provoque un arrêt immédiat du programme avec une erreur ("`cast_error`") en cas d'échec.

5 Points d'évolution

5.1 Renforcer la robustesse des diagnostics ANTLR

Pour éviter des erreurs internes lors d'ambiguïtés de grammaire :

- neutraliser les callbacks "full-context" (journalisation plutôt qu'exception) ;
- améliorer les messages "no viable alternative" par un contexte plus explicite ;
- documenter les constructions à parenthéser pour contourner des ambiguïtés.

5.2 Pistes d'optimisation

Le compilateur actuel produit un code fonctionnel mais naïf. L'ajout d'une passe d'optimisation est nécessaire pour améliorer les performances :

- **Propagation de constantes (Constant Folding)** : Simplifier les expressions arithmétiques et logiques dès la compilation (ex : remplacer `3 + 4` par `7`). Cela peut être implémenté via une méthode `optimize()` dans les nœuds AST.
- **Élimination du code mort** : Supprimer les instructions inaccessibles (après un `return`) ou les branchements inutiles (conditions toujours vraies/fausses).

- **Allocation de registres** : Réduire l'utilisation de la pile en maximisant l'usage des registres disponibles.

6 Bilan de la conception

L'architecture retenue pour ce compilateur privilégie la séparation stricte des préoccupations (*Separation of Concerns*). Le choix d'un AST décoré comme pivot entre les étapes B et C permet une génération de code purement descendante, où chaque nœud dispose de l'intégralité des informations sémantiques nécessaires (types, adresses, index de méthodes).

La robustesse de la conception repose sur trois piliers :

- **Stabilité du typage** : Les trois passes de l'étape B assurent qu'aucune ambiguïté de portée ou de typage ne subsiste avant la génération de code.
- **Abstraction matérielle** : L'utilisation systématique de gestionnaires (**RegManager**, **StackManager**) isole la logique de compilation des contraintes spécifiques de la machine IMA, facilitant un éventuel portage vers une autre architecture.
- **Extensibilité** : La structure modulaire de l'AST permet l'ajout de nouvelles constructions syntaxiques ou de passes d'optimisation sans remettre en cause l'existant.

En conclusion, cette conception offre un compromis optimal entre sécurité d'exécution (via les contrôles dynamiques) et simplicité de maintenance.